技术创新赛道 · 软件开发与应用创新方向

轻排班
——面向中小团队的排班与工时统计工具设计与实现

软件原型与技术研究报告

参赛队伍：〔按主办方报名表填写〕

团队成员：〔按主办方报名表填写〕

指导教师：〔按主办方报名表填写〕

所属学校：〔按主办方报名表填写〕

原型版本 v1.0 · 2026 年 10 月

摘要

中小餐饮、零售及服务业团队普遍缺少专用排班系统，主要依赖纸张或 Excel 管理班次，存在排班冲突靠人工发现、员工可用时间难以记全、全职与兼职工时规则混用、工时汇总繁琐易错等问题。针对上述痛点，本项目设计并实现了轻量级排班与工时统计工具“轻排班”，服务店长（排班负责人）完成员工与班次模板管理、周视图排班编制与调整、排班冲突检测、工时统计和 Excel 报表导出的完整业务流程。

系统采用 Python + Flask + SQLite 技术路线，前端按 Material 3（Material You）设计语言自行实现米 beige 视觉主题；核心的冲突检测与工时计算以纯函数模块实现，支持班次重叠、可用时间冲突、班间休息不足、每日/每周工时上限五类可配置规则，并正确处理跨日班次与统计周期边界。目前项目已完成可运行原型：58 项单元与功能测试全部通过，开发环境实测主要页面响应 P95 不超过 40 毫秒、报表导出 P95 不超过 110 毫秒；演示数据中的三类埋设冲突全部被系统检出。系统已内置管理员密码登录（PBKDF2 加盐哈希、失败限速、可改密）；员工自助查看等扩展功能列为后续计划。

关键词：排班管理；工时统计；冲突检测；Flask；SQLite；Material 3

目录

1. 引言
2. 需求分析
3. 系统设计
4. 系统实现
5. 测试与评估
6. 总结、实施计划与展望
7. 参考文献
8. 附录A 数据库表结构（DDL）
9. 附录B 自动化测试用例清单
10. 附录C 需求验证计划（待开展）
11. 附录D 演示视频脚本要点

注：本报告所述“已完成”均以可运行原型及测试记录为依据；尚未开展的工作在正文中明确标注为“计划”或“后续”。提交格式与模板以主办方正式通知为准。

1 引言

1.1 研究背景

在餐饮、零售及生活服务行业，5～30 人规模的门店数量庞大。此类团队的排班通常由店长或值班经理一人承担：先用 Excel 或纸质表排出下一周班次，再截图发到员工微信群。这一流程能够运转，但高度依赖店长的个人经验与记忆力——每位员工的兼职课时、上课时间、请假习惯都装在“脑子里”，换人就断档。

与排班相伴的工时统计同样依赖手工：月底把每个人的班次逐条相加，扣除班内休息，再区分全职与兼职口径。过程重复、繁琐，且一旦出现跨日班次或临时调班，人工计算极易出错，也难以追溯核对。

1.2 问题描述

通过对手工/Excel 排班流程的逐环节分析，可归纳出四类典型问题：

问题	具体表现	后果
冲突不易发现	同一员工两班时间重叠、与已约定的不可用时间冲突、相邻班次间隔过短等，靠人工逐格核对，排除干扰大且易漏检	班表发布后返工，现场人手冲突或员工临时拒班
约束难以记全	员工可用时间各异（学生兼职尤其明显），约束信息分散在聊天记录中	排班反复修改，沟通成本高
规则口径不一	全职与兼职适用不同的每日/每周工时上限，Excel 无法按用工类型自动区分控制	兼职工时被动超标，机构合规风险上升
汇总繁琐易错	跨日班次、休息扣除、周/月边界等细节需人工处理，月底统计耗时	工时争议缺乏可核对依据

大型企业的人力资源系统覆盖了上述需求，但对小团队而言存在部署重、费用高、功能冗余的问题；通用表格工具又缺乏业务约束。中小团队需要的是一件“趁手的轻工具”。

1.3 研究目标与意义

本项目目标是设计并实现一个面向小规模团队、可本地部署的排班与工时统计工具，完整覆盖“维护员工与班次基础数据 → 编制与调整排班 → 冲突检测与提示 → 工时自动汇总 → 报表导出”的业务闭环。其意义在于：

- **提高排班效率**：周视图网格直接点选排班，常用班次一键套用模板；
- **辅助控制风险**：五类冲突规则自动检测并解释原因，错误级冲突在保存前强制确认；
- **工时可信可核**：统计由排班记录实时推导，口径统一（跨日归属、休息扣除、周期边界均有明确定义），并支持导出 Excel 留档；
- **改善信息获取**：周表一键导出张贴或转发，员工无需翻聊天记录。

1.4 项目范围

本项目面向单店小团队（约 5～50 人），定位为**本阶段可运行、可演示、可检查的技术原型**。范围边界如下：

- **已包含：**员工管理、班次模板、排班编制与调整、冲突检测、工时统计、Excel 导出、规则配置、数据库备份（详见第 2 章需求清单及各条实现状态）；
- **本阶段不包含：**员工账号登录与移动端自助查看、考勤打卡、薪资计算、多门店管理、自动排班算法——均未实现，仅在总结与展望中作为后续工作说明；
- **合规说明：**工时上限、休息间隔等规则在本系统中均为可配置项，随附默认值仅作演示口径；正式使用前，机构应按其所在地区适用的法律法规及自身用工形式确认具体设置，本工具的提示不构成合规结论。

2 需求分析

2.1 用户角色

角色	职责与需求	本阶段状态
店长 / 排班负责人	维护员工档案与班次模板；创建、查看、调整周排班；查看冲突提示并处理；查看工时统计并导出报表；配置排班规则与数据备份	已实现（管理员密码登录，单账号）
普通员工	查看本人排班与工时信息、提交可用时间	本阶段未实现账号体系；员工目前通过导出的周表获取排班信息。员工自助查看已列为后续扩展（见 6.4），不属于已完成功能

2.2 功能需求

编号	需求	说明	状态
FR1	员工信息管理	姓名、岗位、全职/兼职类型、联系方式、可排班日与每日可用时段、在职状态、备注；支持新增、编辑、删除（有排班记录者转为停用以保留历史）	**已实现**
FR2	班次模板管理	班次名称、开始/结束时间（支持跨日）、班内休息分钟、建议岗位、芯片配色；排班时可一键套用	**已实现**
FR3	排班管理	以周为单位浏览排班网格（行=员工、列=周一至周日）；点击空格新增、点击班次芯片编辑/删除；支持上一周/本周/下一周导航	**已实现**
FR4	排班冲突检测	检测班次重叠、可用时间冲突、班间休息不足、每日/每周工时超限五类情况，逐条解释原因；错误级冲突保存前需二次确认（详见 2.3）	**已实现**
FR5	工时统计	按日/周/月汇总每位员工的班次数、排班时长、休息时长、有效工时，含逐日明细；统一处理休息扣除、跨日班次归属与统计周期边界	**已实现**
FR6	报表导出	排班表（日期、星期、员工、岗位、时段、休息、有效工时、备注）与工时统计表（含逐日列与合计行）导出为 .xlsx 文件	**已实现**
FR7	数据与权限管理	数据本地存储于单一 SQLite 文件，支持一键下载备份；系统标题与品牌头像可在设置页自定义（同步用于侧边栏、浏览器标签与站点图标）；冲突规则可在设置页调整。已实现单管理员密码登录（PBKDF2 哈希存储、登录失败限速、设置页改密、登出）；员工自助账号与分角色权限未实现，见展望	部分实现

2.3 冲突检测规则需求

冲突规则全部设计为可配置项（设置页可改，存于数据库 settings 表），默认值仅作演示口径：

编号	规则	级别	判定逻辑与默认值
R1	班次重叠	错误	同一员工两个班次的有效时间区间交集大于 0 分钟即触发，提示重叠时长及对方班次信息
R2	可用时间冲突	错误	排班日不在员工“可排班日”集合内，或班次起止超出“每日可用时段”，即触发；两者均可按员工配置
R3	班间休息不足	提示	同一员工前后两个班次之间间隔低于最小休息间隔（默认 600 分钟 = 10 小时），提示实际间隔
R4	每日工时上限	提示	当日有效工时合计超过按用工类型配置的上限（默认全职 600 分钟、兼职 480 分钟），提示合计值
R5	每周工时上限	提示	本周（周一至周日）有效工时合计超过按用工类型配置的上限（默认全职 2640 分钟 = 44 小时、兼职 1440 分钟 = 24 小时），提示合计值

错误级冲突（R1、R2）阻断保存流程，对话框中给出“返回修改”与“仍要保存”两个出口，避免误操作同时保留门店特殊安排的灵活性；提示级冲突（R3、R4、R5）可直接保存，但在冲突面板与班次芯片上持续可见。为避免同一条件随班次数量重复刷屏，展示层对 R3/R4/R5 按“员工+日期/周期”去重。

2.4 非功能需求

维度	需求（与原型规模相称）
易用性	全中文界面；主要操作（新增班次、调整排班、查看冲突、导出报表）一键可达；表单有明确标签与校验；空状态、操作成功/失败均有可见反馈（提示条、空状态插画位、冲突面板）
响应速度	原型数据规模（数十员工、数百条排班）下，页面与 API 交互应无明显卡顿；以“交互流畅”为验收口径，不承诺生产环境性能指标（实测见 5.3）
数据安全与备份	数据落盘为单一 SQLite 文件；提供一键下载备份；恢复方式文档化（替换文件后重启）；访问需管理员密码登录（PBKDF2 加盐哈希，失败限速 15 分钟 5 次）
隐私保护	最小化收集：不采集员工身份证明、住址、时薪等敏感字段；联系电话选填；全部数据留存本地，不上传第三方
可维护性	核心业务逻辑为不依赖 Web 框架的纯函数模块，配套 58 项自动化测试；数据库结构以 SQL 脚本版本化
可及性	冲突/成功/警告状态同时以图标+文字表达，不单靠颜色；交互元素可见焦点态；排班网格支持键盘操作

3 系统设计

3.1 总体架构

系统采用经典三层结构，核心业务逻辑独立成纯函数模块，与 Web 框架解耦：



图 3-1 系统分层架构（文字示意）

主要数据流：排班页加载时，应用层读取该周排班与相邻班次，调用 `check_conflicts` 计算冲突并在服务端渲染进网格与冲突面板；保存排班时，前端以 JSON 提交，应用层校验后调用同一冲突检测函数，存在错误级冲突则返回 409 与冲突明细，用户确认后携带 force 标志重新提交方可落库。读取与写入共用同一套规则实现，保证“检测口径 = 展示口径”。

3.2 技术选型

技术	版本	选择理由
Python	3.14	语法简洁、标准库完善，适合快速构建原型；团队学习成本低
Flask	3.1	轻量 Web 框架，路由/模板/测试支持齐备，无需脚手架即可运行，匹配“轻量单店工具”定位
SQLite	3（标准库 sqlite3）	零配置、单文件、事务完整；单店单机场景无需独立数据库服务，备份即复制文件
Jinja2 + 原生 CSS/JS	3.1 / ES2020	服务端渲染减少前后端联调成本；Material 3 无官方轻量 Web 组件且比赛环境可能离线，故按 M3 设计令牌（token）自行实现组件样式，不引入构建工具与运行时依赖

openpyxl	3.1	纯 Python 写入 .xlsx，支持合并单元格、填充色等样式，满足报表美化需求
pytest	9.1	参数化与夹具机制成熟，适合对纯函数核心做密集单元测试

> 选型限制说明：Flask 开发服务器为单线程参考实现，原型按单用户设计，未做并发控制与生产部署加固；若后续多用户使用，应更换生产级 WSGI 服务器并增加认证层。

3.3 数据库设计

共 4 张表：员工（employees）、班次模板（shift_templates）、排班记录（schedules）、规则配置（settings）。完整 DDL 见附录 A。

表	主要字段	说明
employees（员工）	id(PK)、name、position、employment_type(fulltime/parttime)、phone、available_days(JSON 数组，ISO 星期 1–7)、available_from/to、status(active/inactive)、note、created_at	可用时间用于冲突检测；时薪等敏感字段按最小化收集原则未纳入
shift_templates（班次模板）	id(PK)、name、start_time、end_time(早于开始即跨日)、break_minutes、position、color(芯片配色 token)、note	常用班次的可复用定义
schedules（排班记录）	id(PK)、employee_id(FK→employees)、work_date(开始日期)、start_time、end_time、break_minutes、position、template_id(FK→shift_templates, ON DELETE SET NULL)、note、created_at	班次时间在创建时保存快照，模板删除不影响历史排班；索引 (employee_id, work_date) 与 (work_date) 支撑周查询
settings（规则配置）	key(PK)、value	门店名称、最小休息间隔、四项工时上限等键值对，界面可改

实体关系： employees 1—N schedules；shift_templates 1—N schedules（弱关联，置空不级联删除）。

设计决策一（工时不单独建表）： 工时统计由排班记录实时推导而非落库。排班是工时的唯一数据源，推导式统计天然避免“排班已改、工时未同步”的双写不一致；若后续接入考勤打卡，再引入实际工时表与“计划/实际”对照，属扩展方向。

设计决策二（跨日归属）： 跨日班次整体归属开始日期所在的日/周/月统计周期。该规则在冲突检测（周上限）与工时统计两处一致执行，并在界面文案中明示。

3.4 核心模块设计

3.4.1 排班模块

排班编制围绕周视图网格展开：行=在职员工（含岗位与用工类型），列=周一至周日。新增班次流程为：点击空白格 → 对话框自动带入员工与日期 → 选择模板自动填充时段与休息（或自定义时间）→ 前端基础校验 → 提交 JSON API → 服务端校验与冲突检测 → 无错误级冲突直接保存，有则返回 409 由用户确认。调整与删除通过点击既有班次芯片进入同一对话框完成，编辑时冲突检测自动排除自身记录。

3.4.2 冲突检测模块

检测函数 `check_conflicts(候选排班, 该员工既有排班, 员工约束, 规则配置)` 为纯函数，输入输出均为值对象，判定步骤：

1. 把“日期 + 起止时间”展开为绝对时间区间（结束早于等于开始视为次日结束），计算候选班次的有效工时；
2. **R1 重叠**：对每条既有班次求区间交集，交集 > 0 分钟即产出错误级冲突，消息含对方班次与重叠分钟数；
3. **R2 可用时间**：候选日期的 ISO 星期不在集合内，或起止超出每日可用时段，产出错误级冲突；
4. **R3 班间休息**：分别找候选班次之前最近结束、之后最近开始的班次（跳过已判重叠者），间隔低于配置值即产出提示；
5. **R4/R5 工时上限**：按“开始日期”归集当日/当周（周一至周日）全部既有班次的有效工时，加候选值后与按用工类型选取的上限比较，超出即产出提示；
6. 返回冲突列表（规则、级别、含具体数字的中文消息、关联班次 id），由页面/API 层决定渲染方式与是否阻断。

3.4.3 工时统计模块

统计口径统一为：**有效工时 = 班次跨度 − 班内休息**（跨度按跨日规则展开）；**跨日班次整体归属开始日期所在周期**；周定义为周一至周日，月为自然月。汇总函数对区间内排班逐条累加，输出每人的班次数、排班时长、休息时长、有效工时与逐日明细（用于周视图的逐日列与导出）。统计与冲突检测共用同一组时间函数，口径一致。

3.4.4 导出模块

导出在服务端用 openpyxl 生成工作簿并内存返回。字段映射：

报表	列结构	样式与辅助信息
排班表	日期、星期、员工、岗位、班次时段、休息(分钟)、有效工时(小时)、备注	标题行含门店名与周期；存在冲突的排班在备注列追加“冲突提示 N 条”
工时统计表	员工、岗位、类型、班次数、排班时长、休息时长、有效工时 + 周视图追加周一至周日逐日列	末行为合计行；表头使用主色容器色填充，冻结表头

3.5 界面设计（Material 3 米 beige）

界面遵循 Material 3 设计语言的语义化色彩角色与组件形态，以米 beige 为主色调，营造清晰、亲和的日常工具气质。全部色彩以 CSS 自定义属性（设计令牌）统一管理：

语义角色	色值	应用位置
页面背景 background	#F7F3EA	全局页面底色
表面/卡片 surface	#FFFDF8	卡片、表格、对话框
主色 primary	#78664F	填充按钮、选中导航、今日列强调
主色容器 primary-container	#E9DDC8	表头、选中态背景、班次芯片（早班）
次要色 secondary	#6D7765	兼职芯片、次要容器
强调色 tertiary	#B47B55	晚班芯片、强调元素
主要文字 / 次要文字	#292720 / #68645C	正文与辅助说明
边框 outline	#DED7CA	卡片描边、分隔线、表格线
成功 / 警告 / 错误	#526B55 / #94651E / #A1463D	状态芯片与冲突面板，均配图标+文字表达

组件层面实现了 M3 风格的导航栏（导航项胶囊选中态）、卡片、填充/色调/描边/文本四类按钮、对话框、提示条（snackbar）、芯片（chip）、分段按钮与数据表格。排班网格以间距与表面层级组织信息，不使用厚重阴影；今日列以主色高亮；跨日班次以“夜班”配色芯片区分。所有状态（冲突、成功、警告）同时通过图标与文字表达；交互元素均有可见焦点态，排班网格单元格与班次芯片可键盘到达（Tab + Enter/Space 触发）。

页面共 6 个：概览（今日排班+本周冲突摘要+统计卡片）、排班（周网格+冲突面板+班次对话框）、员工、班次模板、工时统计（按日/周/月切换）、设置（品牌外观+规则配置+数据管理）。界面同时兼顾小屏浏览（导航栏自动收窄为图标栏，网格横向滚动），小屏效果见 4.5 节图 4-8。

4 系统实现

4.1 开发与运行环境

项	内容
操作系统	Windows 11（x64）；代码不含平台特有依赖，亦可运行于 Linux/macOS
开发语言与框架	Python 3.14.7；Flask 3.1.3；openpyxl 3.1.5；pytest 9.1.1
运行环境	任意现代浏览器（Chrome / Edge / Firefox）；无需联网，无构建步骤
部署形态	本机运行 `python app.py`，监听 127.0.0.1:5001；数据文件 instance/shiftlite.db；亦提供 Dockerfile 与 SHIFTLITE_DB/SHIFTLITE_SECRET 环境变量用于云端部署（见 README）

4.2 项目结构

```
shiftlite/
├── app.py              # Flask 应用：页面路由、JSON API、导出与备份端点
├── db.py               # SQLite 连接、建库初始化、规则配置读写
├── schema.sql          # 数据库 DDL（附录 A）
├── seed.py             # 演示数据（10 员工 / 4 模板 / 两周 68 条排班，含埋设冲突）
├── core/               # 核心业务（纯函数，不依赖 Flask）
│   ├── scheduling.py   #   时间区间展开、有效工时、五类冲突检测
│   ├── worktime.py     #   按日/周/月工时汇总
│   └── exporter.py     #   openpyxl 报表生成
├── templates/          # Jinja2 页面（base + 6 页面 + 宏）
├── static/
│   ├── css/theme.css   # Material 3 米 beige 主题（设计令牌 + 组件）
│   └── js/             # app.js 公共交互 / schedule.js 排班页交互
├── tests/              # pytest：26 项核心单元 + 6 项工时 + 3 项导出 + 6 项认证 + 17 项功能
├── scripts/benchmark.py # 性能计时脚本
└── docs/               # 技术研究报告、截图、性能测量结果
```

4.3 启动方法

```
pip install -r requirements.txt   # 安装依赖（Flask、openpyxl；开发需 pytest）
python seed.py                    # 首次运行写入演示数据（python seed.py --force 可重置）
python app.py                     # 启动服务，浏览器访问 http://127.0.0.1:5001（默认密码 shiftlite
123，登录后请修改）
python -m pytest tests/           # 运行全部自动化测试
```

4.4 核心代码说明

4.4.1 时间区间展开与有效工时（core/scheduling.py）

```
def shift_span_minutes(start_hhmm: str, end_hhmm: str) -> int:
    """班次跨度（分钟），跨日自动加 24 小时；起止相同视为非法。"""
    start, end = parse_hhmm(start_hhmm), parse_hhmm(end_hhmm)
    if start == end:
        raise ValueError("班次开始与结束时间相同")
    span = end.hour * 60 + end.minute - (start.hour * 60 + start.minute)
    if span < 0:
        span += 24 * 60
    return span

def effective_minutes(start_hhmm, end_hhmm, break_minutes) -> int:
    """有效工时（分钟）= 跨度 − 休息时长，下限 0。"""
    return max(0, shift_span_minutes(start_hhmm, end_hhmm) - int(break_minutes or 0))

def shift_datetime_range(work_date, start_hhmm, end_hhmm):
    """把“日期 + 起止时间”展开为绝对时间区间，跨日结束时间落到次日。"""
    start = datetime.combine(work_date, parse_hhmm(start_hhmm))
    end = datetime.combine(work_date, parse_hhmm(end_hhmm))
    if end <= start:
        end += timedelta(days=1)
    return start, end
```

这三个函数是全部口径的基础：跨度计算允许“19:00–01:00”式跨日班次；有效工时统一扣除休息；区间展开把班次映射到绝对时间轴，供重叠与间隔判定使用。起止相同被显式拒绝，避免出现“24 小时班次”的二义性。

4.4.2 冲突检测（core/scheduling.py，节选）

```
# 规则一：班次时间重叠（硬冲突）
for s, e, row in spans:                    # spans 为既有班次的绝对时间区间
    ov = overlap_minutes(c_start, c_end, s, e)
    if ov > 0:
        conflicts.append(Conflict(
            "overlap", SEVERITY_ERROR,
            f"与已有班次（{row['work_date'].strftime('%m月%d日')} "
            f"{row['start_time']}-{row['end_time']}）重叠 {ov} 分钟",
            row.get("id")))

# 规则五：每周（周一至周日）有效工时上限，按班次开始日期归属（提示）
w_start, w_end = week_range(c_date)
week_rows = [row for row in existing
             if w_start <= row["work_date"] <= w_end
             and (candidate.get("id") is None or row.get("id") != candidate["id"])]
week_total = c_eff + sum(
    effective_minutes(r["start_time"], r["end_time"], r["break_minutes"])
    for r in week_rows)
weekly_limit = int(settings.get(
    "weekly_max_minutes_fulltime" if emp_type == "fulltime"
    else "weekly_max_minutes_parttime") or 0)
if weekly_limit > 0 and week_total > weekly_limit:
    conflicts.append(Conflict(
        "weekly_limit", SEVERITY_WARNING,
        f"本周（{w_start.strftime('%m月%d日')}-{w_end.strftime('%m月%d日')}）"
        f"有效工时合计 {_fmt_hours(week_total)}，超过{type_label}每周上限 {_fmt_hours(weekly_limi
t)}"))
```

检测函数返回统一的 `Conflict` 数据对象（规则、级别、解释性消息、关联记录），消息直接给出“与哪一天哪一班的重叠分钟数”“合计多少小时、超哪条上限”等具体数字，保证提示可理解、可核对。页面/API 层据此决定阻断（错误级）或仅提示（提示级）。

4.4.3 冲突确认的 API 流程（app.py）

```
conflicts = _conflicts_for(conn, payload)
hard = [c for c in conflicts if c["severity"] == "error"]
if hard and not request.args.get("force"):
    return jsonify({"conflicts": conflicts}), 409   # 前端弹出冲突确认
cur = conn.execute("INSERT INTO schedules(...) VALUES (...)", (...))
conn.commit()
return jsonify({"id": cur.lastrowid, "conflicts": conflicts}), 201
```

保存接口遇到错误级冲突返回 409 与完整冲突明细；前端对话框内联展示冲突列表，用户选择“仍要保存”时携带 force 标志重新提交。该设计既强制人工确认硬冲突，又保留门店特殊安排的最终决定权。

4.4.4 Excel 导出（core/exporter.py，节选）

```
def export_hours(store_name, period_label, summary, day_columns=None):
    wb = Workbook(); ws = wb.active; ws.title = "工时统计"
    headers = ["员工", "岗位", "类型", "班次数", "排班时长(小时)",
               "休息时长(小时)", "有效工时(小时)"]
    if day_columns:
        headers += [label for _, label in day_columns]      # 周一…周日逐日列
    ...
    total = ["合计", "", "", sum(i["shifts"] for i in summary), ...]  # 合计行
    for col, value in enumerate(total, start=1):
        cell = ws.cell(row=r, column=col, value=value)
        cell.fill = PatternFill("solid", fgColor=THEME["total_fill"])
```

4.5 功能实现情况与界面

以下截图均为实际运行系统（演示数据）的真实画面。

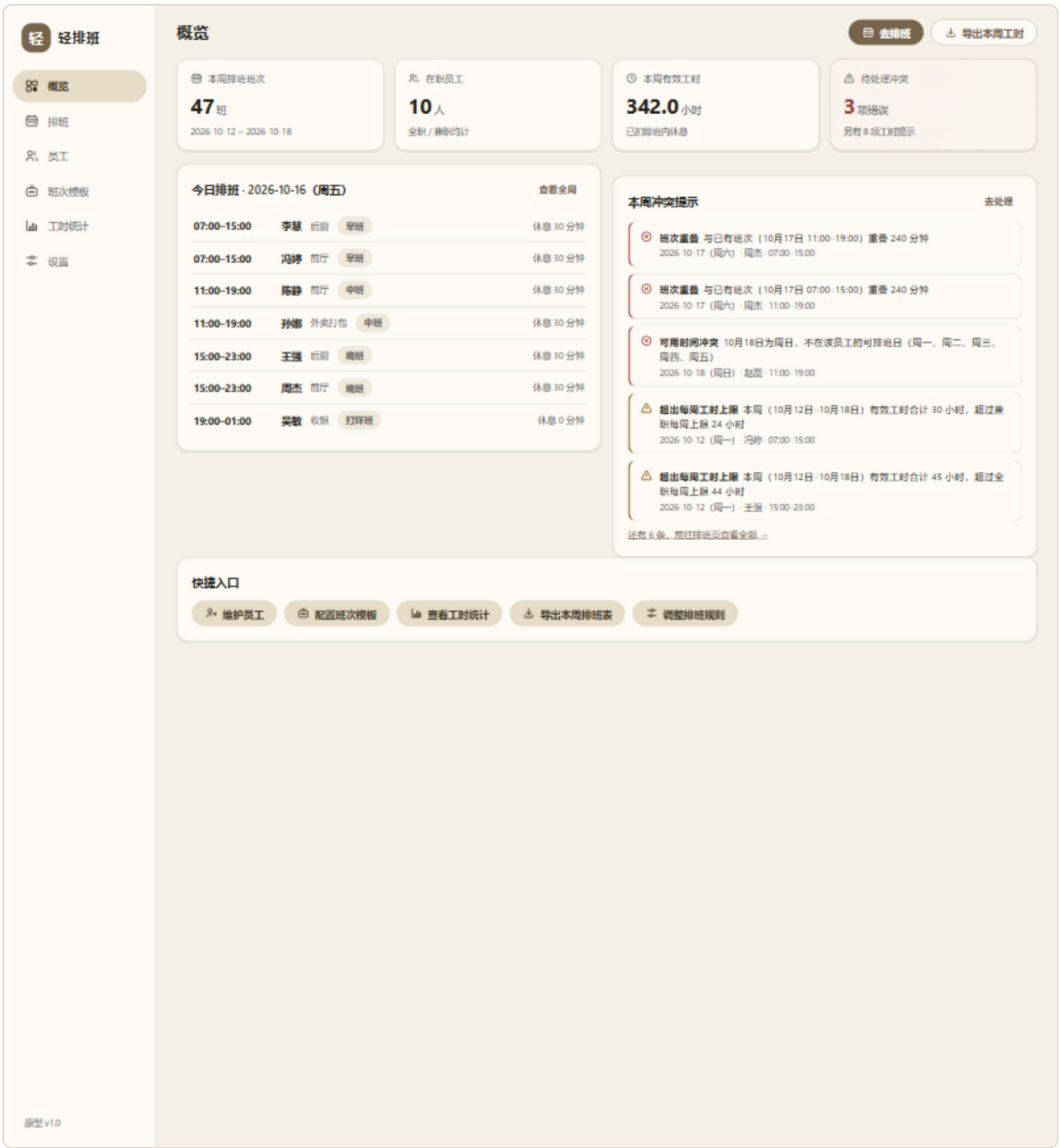


图 4-1 概览页：本周统计卡片、今日排班与冲突摘要（右上可直达排班与导出）



图 4-2 排班页周视图：员工 × 周一至周日网格，班次芯片按模板配色，含冲突者显示警示角标（周五/周六可见重叠冲突标记）

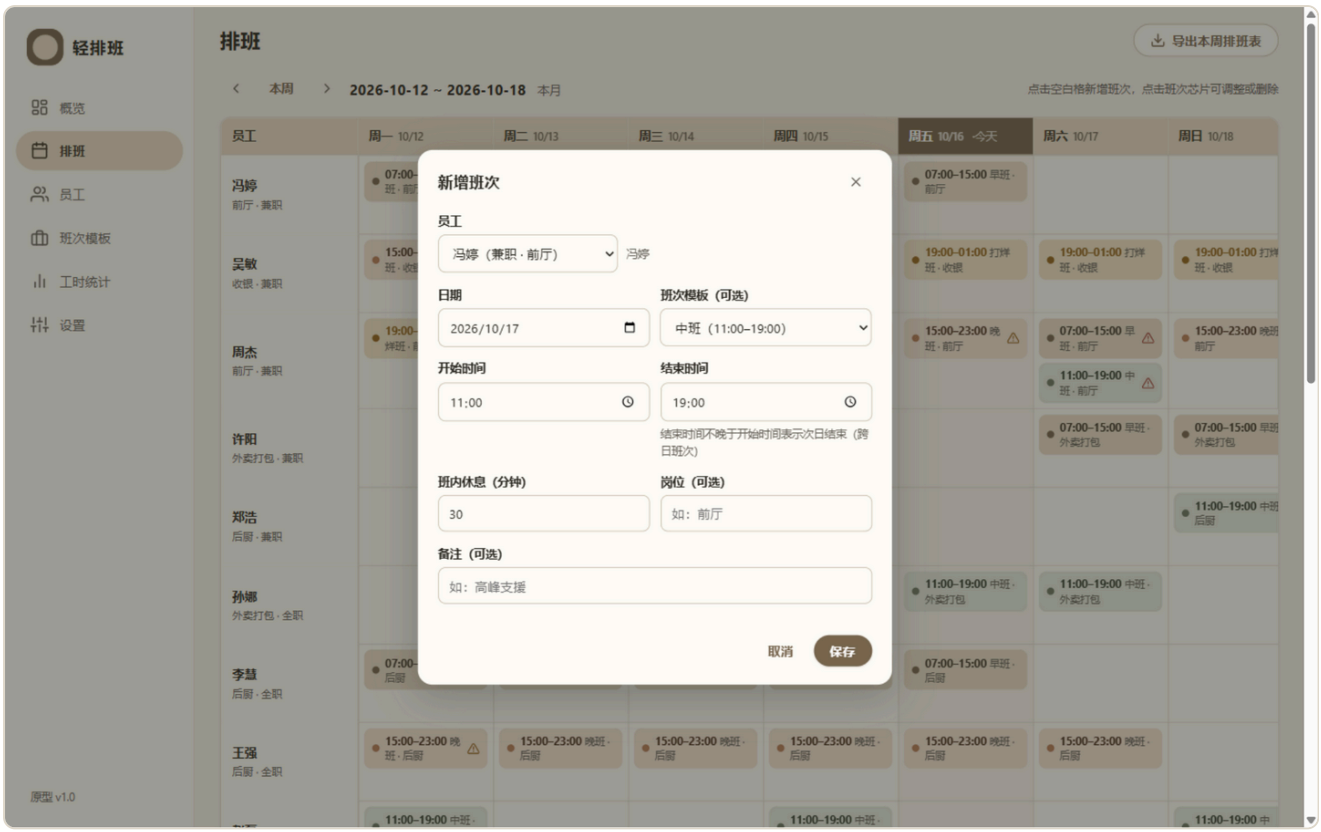


图 4-3 新增班次对话框：套用“中班”模板自动填充时段与休息，支持跨日说明

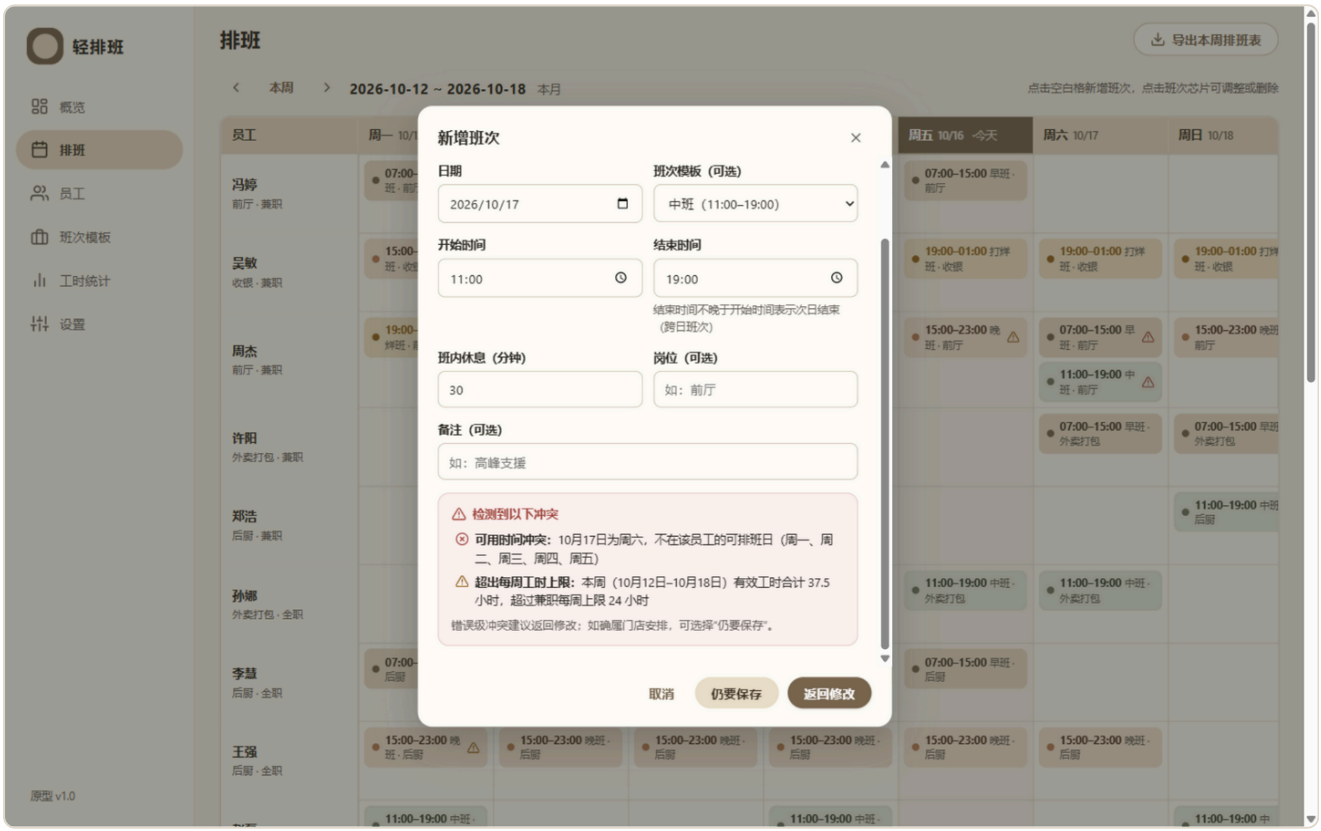


图 4-4 冲突拦截演示：为周六休班的员工排中班，系统给出“可用时间冲突”（错误级）与“每周工时上限”（提示级），需“返回修改”或“仍要保存”二次确认

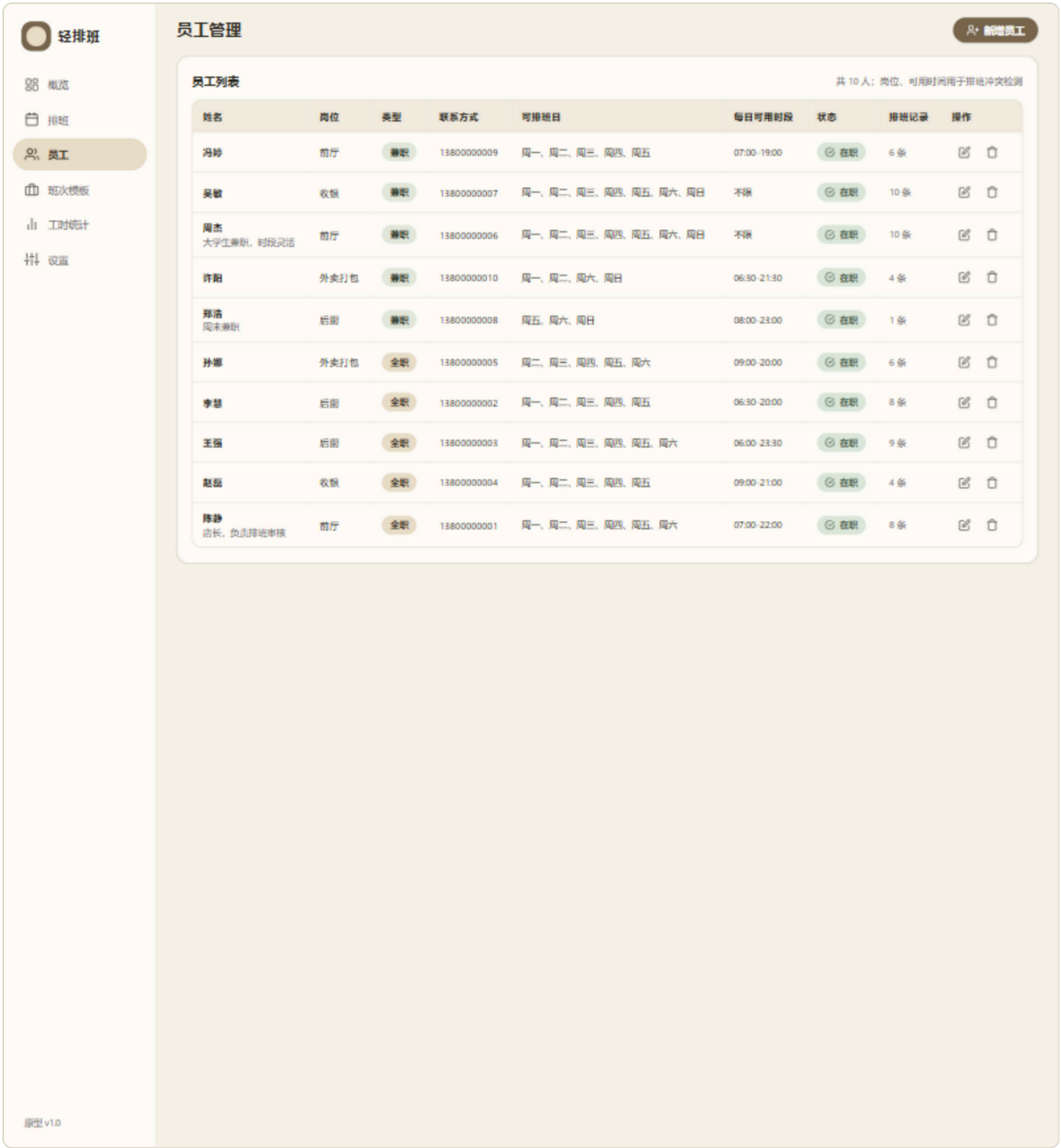


图 4-5 员工管理：用工类型、可排班日与可用时段是冲突检测的数据来源；有排班记录的员工删除时转为停用以保留历史

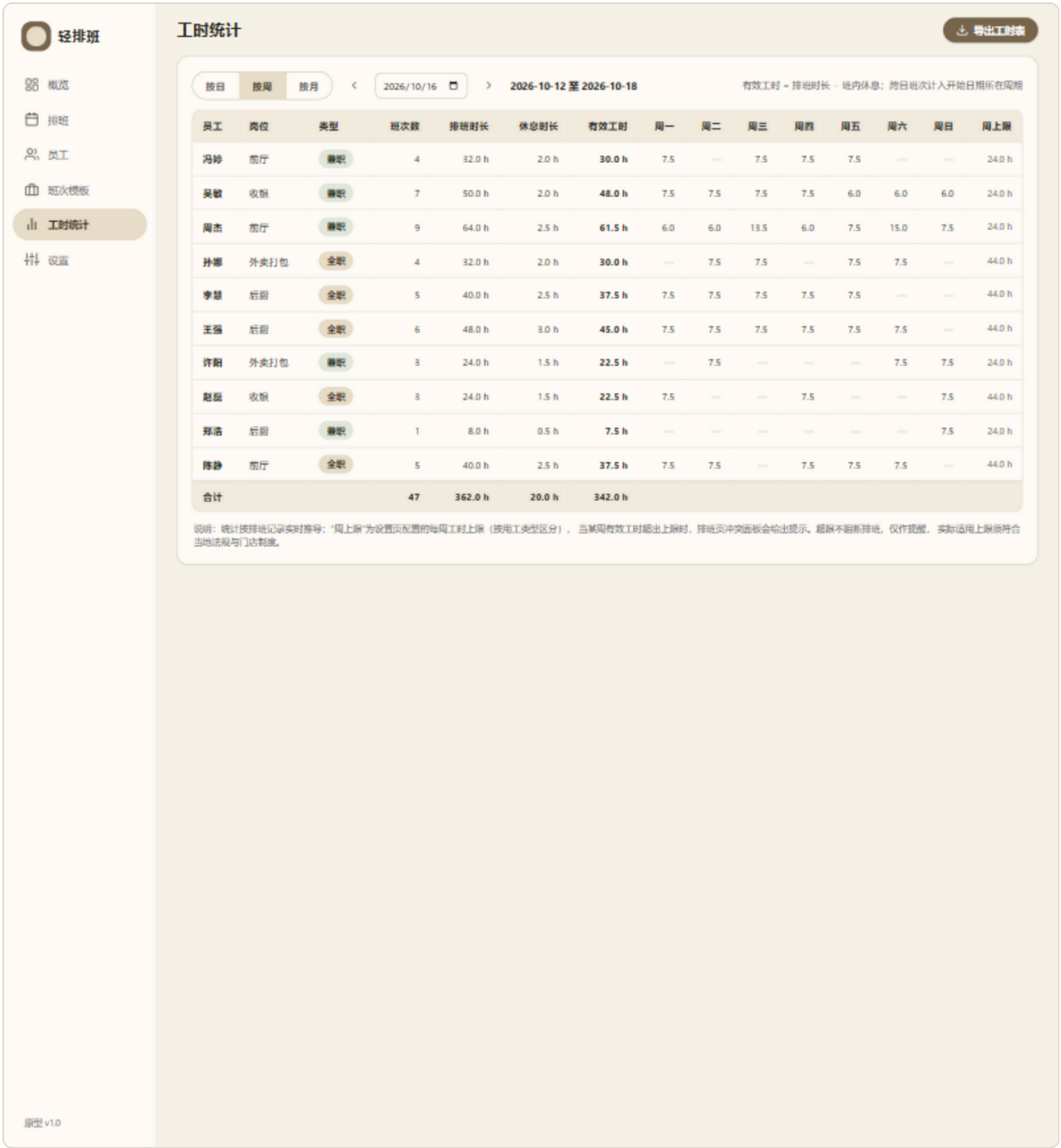


图 4-6 工时统计（按周）：逐日列与合计行，口径说明（有效工时=排班时长−休息，跨日计入开始日期所在周期）随页展示

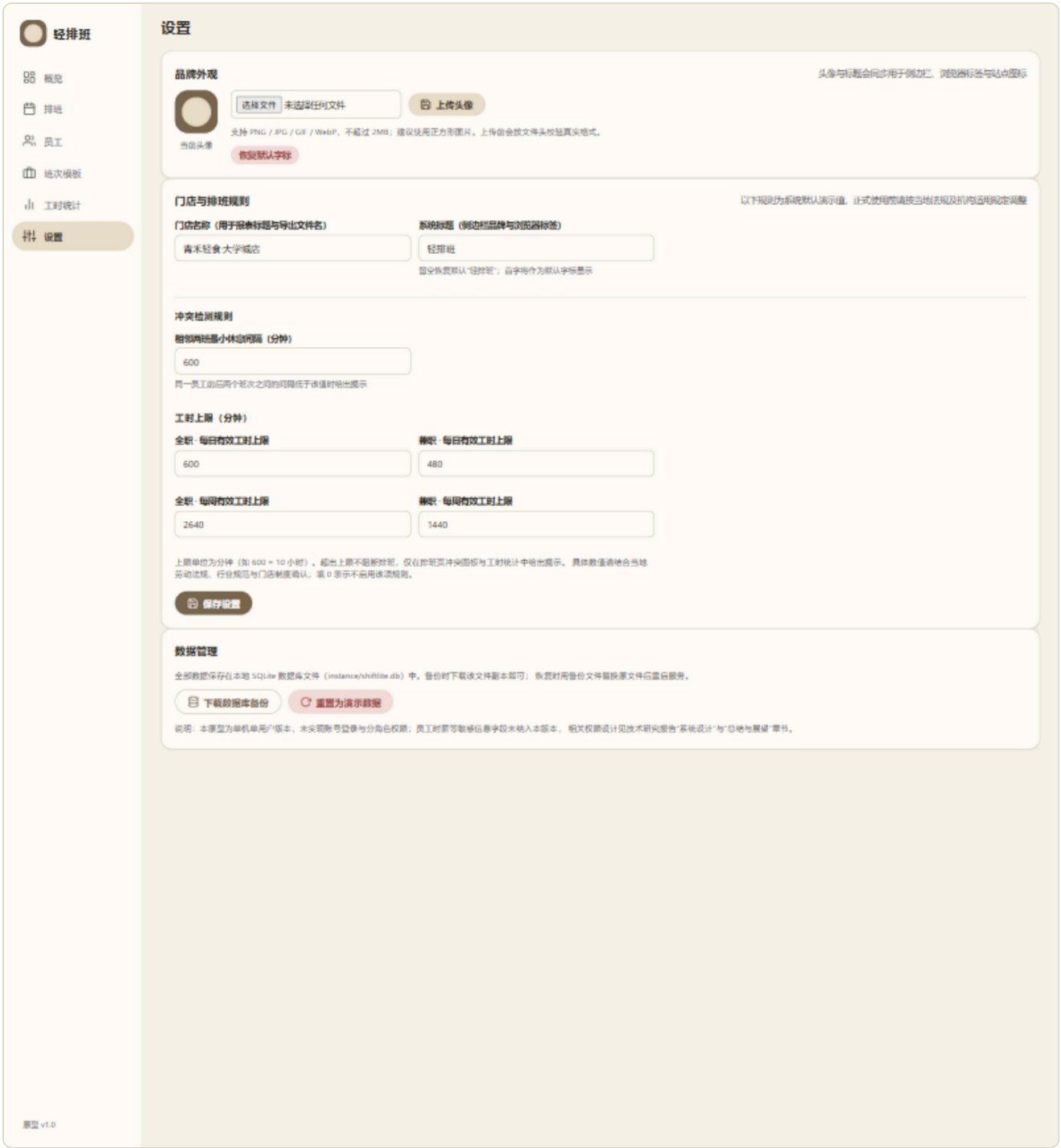


图 4-7 设置页：品牌外观（系统标题与头像，含格式与大小校验）、五类冲突规则全部可配置并明示“须按当地法规与机构制度确认具体设置”；数据管理提供备份下载与演示数据重置



图 4-8 小屏（390px）适配：导航收窄为图标栏，卡片纵向堆叠

各功能与需求条目的对应关系及实现位置：

需求	实现要点	对应代码 / 界面

FR1 员工管理	增删改查、可用时间维护、停用保护	app.py employees_page；templates/employees.html（图 4-5）
FR2 班次模板	跨日定义、休息分钟、配色、使用计数	app.py shift_templates_page；shift_templates.html
FR3 排班维护	周网格点选、模板套用、编辑/删除	schedule_page + /api/schedules；schedule.js（图 4-2、4-3）
FR4 冲突检测	五类规则、409 确认流、面板去重	core/scheduling.py；图 4-4
FR5 工时统计	日/周/月切换、逐日明细、口径注释	core/worktime.py；hours.html（图 4-6）
FR6 报表导出	排班表/工时表 xlsx、主题样式、冲突备注	core/exporter.py；/export/* 端点
FR7 数据与权限	SQLite 单文件、备份下载、规则配置	settings_page、/backup（图 4-7）

4.6 实现状态清单

条目	状态
员工 / 模板 / 排班 / 冲突 / 工时 / 导出 / 规则配置 / 备份	**已实现并通过测试**
品牌自定义（系统标题、头像上传/移除，同步站点图标）	**已实现并通过测试**
小屏适配、键盘可达、状态反馈（空/成功/失败/冲突）	**已实现**
管理员密码登录（PBKDF2 哈希、限速、改密、登出）	**已实现并通过测试**
员工自助账号与分角色权限	未实现，列为后续工作（6.4）
员工移动端自助查看、考勤打卡、排班建议算法、薪资对接	未实现，列为后续工作（6.4）

5 测试与评估

5.1 测试方法与环境

测试在开发机（Windows 11，Python 3.14.7）上进行，包含四类：

- **单元测试（pytest）**：针对不依赖 Web 框架的核心模块（时间计算、冲突检测、工时汇总、Excel 生成），每项测试以独立临时数据库运行；
- **功能测试**：基于 Flask test client 的端到端请求验证（页面渲染、增删改查、冲突拦截与强制保存、校验拒绝、导出内容字节级检查）；
- **界面验证**：浏览器实际操作与截图核对（本章截图即来源于此）；
- **性能计时**：对运行中的服务以脚本连续重复调用并统计耗时（5.3）。

全部 58 项自动化测试于 2026-10-21 重新执行，结果 **58 通过 / 0 失败**（用时 13.83 秒）。完整用例清单见附录 B。

5.2 测试用例与结果

模块	用例数	通过	覆盖内容
时间计算与冲突检测（test_scheduling.py）	26	26	跨度/有效工时/跨日展开、周月边界、五类规则的触发与不触发、级别区分、编辑排除自身、非法输入
工时统计（test_worktime.py）	6	6	单人/多人汇总、休息扣除、跨日归属、周界（周日夜班）、月界、小时换算
Excel 导出（test_exporter.py）	3	3	排班表内容、工时表逐日列与合计、空数据健壮性
认证与账号（test_auth.py）	6	6	未登录重定向与 API 401、登录页与错误密码、登录跳转 next 与登出、改密后旧密码失效、改密三类校验、失败限速
端到端功能（test_app.py）	17	17	页面渲染、员工/模板增删改校验、冲突 409→强制保存、演示数据冲突检出、停用员工隐藏、设置回写、备份下载
合计	**58**	**58**	

与竞赛要求场景直接对应的代表性用例（全部实际执行通过）：

场景	输入 / 操作	预期	结果
正常排班	为可用员工在无班时段排班（API/界面）	201 保存成功，网格出现班次芯片	**通过**
班次重叠	同一员工周六已排 07:00–15:00 与 11:00–19:00，再排 11:30–14:00	返回 409，冲突含“重叠 240 分钟”，保存被拦截	**通过**

不可用时间	为“仅周一至周五可用”的员工排周六中班	错误级冲突：“10月17日为周六，不在该员工的可排班日（周一…周五）”	**通过**
工时上限（日）	兼职员工当日累计超上限	提示级冲突，消息含当日合计与上限值，可保存但面板可见	**通过**
工时上限（周）	周内累计超 24 小时（兼职口径）	提示级冲突，消息含本周区间与合计；同一员工同周只提示一次	**通过**
跨日班次重叠	19:00–01:00 与次日 00:30–04:30	判定重叠（按绝对时间区间）	**通过**
跨日工时归属	周日 21:00–次日 01:00	计入周日所在周（周一至周日），下一周统计为 0	**通过**
休息扣除	09:00–17:00 休息 30 分钟	有效工时 7.5 小时；导出工时表逐日列同值	**通过**
统计周期边界	区间两端外的排班记录	不计入所选周期汇总	**通过**
报表导出	GET /export/schedule 与 /export/hours	返回 xlsx（MIME 正确）；表头、逐日列、合计行内容与页面一致	**通过**
非法输入	开始时间 99:00、起止相同等	400 并返回可读的中文错误列表，不落库	**通过**

5.3 性能评估

以脚本对运行中的原型连续重复调用（读操作 20 次、写操作 10 次，写后即删恢复数据规模），记录服务端响应耗时。测量环境：Windows 11 / Python 3.14.7 / Flask 开发服务器（单线程）/ SQLite 本地文件 / 回环网络；数据规模：10 名在职员工、4 个班次模板、68 条排班记录（当前周 47 条）、当前周冲突 11 条。

操作	次数	平均 (ms)	P95 (ms)	最大 (ms)
概览页 GET /	20	13.4	18.7	—
排班页（周视图+冲突面板）GET /schedule	20	26.7	38.6	—
工时统计页（周）GET /hours	20	5.6	11.5	—
员工页 GET /employees	20	6.1	8.1	—
冲突检测 GET /api/conflicts（本周全部记录）	20	6.9	12.0	—
导出排班表（xlsx）	20	92.9	109.3	—
导出工时表（xlsx）	20	64.1	73.0	—
新增排班 POST /api/schedules（含冲突检测）	10	8.4	12.3	—

结论：在原型设计规模下，全部页面与交互响应在 40 毫秒以内、报表导出在 110 毫秒以内，交互无感知延迟，满足 2.4 节“交互流畅”的验收口径。本表为开发环境机器计时，用于验证原型规模下的响应速度，不构成生产环境性能承诺；并发压力测试未开展，列为后续计划。

5.4 冲突识别有效性评估

演示数据在当前周刻意埋设了三类错误级冲突（同一员工周六两班重叠 240 分钟、两名员工被排到不可排班日）以及多组提示级场景（班间休息 0/8 小时、兼职与全职触及每日/每周上限）。系统在冲突面板共检出 11 条（错误 3 条、提示 8 条），逐条含具体数字与关联班次，5.2 节用例与图 4-4 交互流程互为印证。需说明：该评估基于构造数据验证“规则是否按定义执行”，不等同于真实门店数据的漏检率统计——后者依赖实地数据，见附录 C 计划。

5.5 易用性评估

本项目按可用性启发式对原型逐项自查：

检查项	现状
系统状态可见	操作成功/失败均有提示条；冲突即时出现在面板与芯片角标；空数据页有空状态引导
防错与恢复	表单校验前置（时间格式、起止相同、越界）；错误级冲突强制二次确认；删除班次有确认框且可重新添加恢复
一致性	同一套按钮/芯片/对话框样式贯穿 6 个页面；日期、时间、工时格式统一
效率	模板一键套用；周导航与“本周”快捷键位；概览页直达高频操作
可及性	状态不只靠颜色（图标+文字）；焦点可见；网格支持键盘操作

局限：以上为开发者自查结论，尚未开展真实用户参与的任务计时、满意度问卷或与 Excel 流程的对照实验；相关测试条件、对象与指标已在附录 C 中作为需求验证计划列出，本报告不虚构此类对比数据。

5.6 已知简化与局限

- **重叠班次工时重复计入：**当错误级重叠已存在时，日/周工时按记录逐条累加，重叠时段会被重复计入合计；系统通过 R1 规则显著标出重叠以便人工核实，未做去重合并；
- **单账号、无并发控制：**已实现管理员密码登录（会话 7 天），但按单店长使用设计，无多账号与并发写保护；
- **无操作审计：**调班、删除暂无历史留痕（数据库备份可作时点恢复）；
- **开发服务器：**未做生产部署加固（HTTPS、WSGI 服务器、进程守护）；
- **可用时段仅支持“每日一段”窗口：**不支持同一员工一天内多段可用时间，跨日班次与窗口组合存在场景限制（如跨日班次必超当日窗口，需为夜班员工留空时段约束）。

6 总结与展望

6.1 工作总结

本项目完成了一个面向中小团队排班场景的完整技术原型，主要工作包括：

- **需求层面**：从手工/Excel 排班的四个具体痛点出发，定义了 7 项功能需求、5 类可配置冲突规则与相称的非功能需求，明确了本阶段范围边界；
- **设计层面**：完成三层架构与 4 表数据库设计；确立“排班记录为工时唯一数据源”“跨日归开始日期”“错误级阻断/提示级可见”三项关键设计决策；按 Material 3 规范建立米 beige 设计令牌体系并落地为可复用组件样式；
- **实现层面**：交付可运行的 Flask + SQLite 原型（约 3400 行源码），覆盖员工管理、班次模板、周视图排班、五类冲突检测、日/周/月工时统计、Excel 报表导出、规则配置、备份与管理员密码登录；核心逻辑以纯函数实现，与框架解耦；
- **验证层面**：58 项自动化测试全部通过（含登录保护、错误密码、改密校验、限速等认证场景），覆盖正常排班、重叠、不可用时间、工时上限、跨日、休息扣除、周期边界与导出等竞赛要求场景；以脚本实测原型规模下页面响应 P95 ≤ 40 毫秒、导出 P95 ≤ 110 毫秒；
- **成果物**：可运行软件与源代码、技术研究报告（本文）、9 张真实运行截图、性能测量记录、演示视频脚本（附录 D）。

6.2 不足

原型仍有明显局限：单管理员账号、无员工自助与分角色权限；重叠时段工时重复计入；无操作审计；可用时段仅支持每日一段；未开展真实用户评估与并发测试。上述已在 5.6 节如实说明，均不影响本阶段功能验证的有效性。

6.3 项目实施与团队计划

为使本报告同时具备项目书所需的实施信息，以下内容按当前原型状态编制；团队姓名与学校等身份信息须在提交前依据报名表补齐。

角色	成员（待填写）	主要职责与成果
项目负责人	〔姓名〕	需求边界、版本计划、材料统筹、最终验收与答辩协调
后端与算法	〔姓名，可与负责人相同〕	数据库、排班接口、冲突检测、工时统计及自动化测试
前端与交互	〔姓名〕	页面结构、周视图排班交互、响应式适配与可用性检查
测试与材料	〔姓名〕	测试记录、性能测量、演示数据、报告与演示视频
指导教师	〔姓名〕	技术路线、研究规范、应用边界及竞赛材料指导

同一成员承担多个角色时可合并填写，但应保留具体工作和可核验成果；成员顺序、负责人和指导教师须与报名系统一致。

6.3.1 实施进度

阶段	状态 / 计划	主要任务与验收物
需求与原型	**已完成**	明确用户、业务流程、7 项功能需求与 5 类冲突规则；形成可运行原型
核心功能与验证	**已完成**	完成排班、冲突检测、工时统计、导出、登录与备份；58 项自动化测试通过
竞赛材料定稿	提交前完成	补齐团队与学校信息，核对官方模板、文件命名、签字盖章和演示视频要求
真实用户验证	后续计划	按附录 C 邀请有排班经验者开展效率对照、提示可读性与规则口径验证
试点与迭代	验证通过后	选择单门店试点，增加审计、员工端和生产部署能力，再评估多门店推广

6.3.2 应用推广计划

1. **校内或熟悉门店验证**：以匿名演示数据完成首次培训和任务测试，收集排班负责人反馈；
2. **单店试点**：在获得经营方书面许可后导入脱敏数据，连续运行 2～4 周，对比人工流程的耗时和冲突残留；
3. **标准化交付**：形成安装、备份恢复、规则配置与数据迁移清单，提供本地部署和 Cloudflare Workers 两种形态；
4. **范围扩展**：在试点证据支持下增加员工自助与权限审计，再考虑多门店和第三方系统对接。

6.3.3 资源与预算需求

项目	当前需求	说明
软件与开发工具	0 元	使用 Python、Flask、SQLite、openpyxl 等开源组件
本地演示环境	利用现有设备	普通 Windows 电脑和现代浏览器即可运行
云端试用	免费额度起步	可使用 Cloudflare Workers + D1 免费额度；正式运营费用按实际访问量核算
用户验证与差旅	〔按实际填写〕	问卷、打印、交通或访谈补贴须按真实支出和赛事规则填报
域名及生产运维	〔按方案填写〕	进入公开试点后再根据域名、备份和服务器方案列支

本表仅为资源规划，不虚构未发生支出；如主办方提供统一预算表，应将最终金额迁移至官方模板。

6.4 后续工作

方向	说明

员工自助查看	增加员工账号（可先用“手机号+验证码”的轻量方案），员工在移动端查看本人周排班与工时，提交可用时间与请假；排班变更推送提醒
权限完善	在现有管理员密码登录基础上扩展员工账号与分角色访问控制（店长/副店/普通员工）；字段级权限（如时薪字段仅店主可见）；操作审计日志
考勤对照	引入实际打卡记录表，形成“计划排班 vs 实际工时”对照视图与差异报告；工时口径保持与本期一致
数据可靠性	自动定时备份与一键恢复；SQLite 迁移至生产级部署（WSGI 服务器 + HTTPS）；多门店支持
排班建议	在冲突检测基础上增加候选推荐：按员工可用时间、工时均衡度与历史偏好生成草稿班表，仍由店长确认——定位为辅助而非替代人工决策
薪资对接	本工具不内置薪资计算；通过标准化导出口径（工时表字段已固定）与薪酬系统对接，避免两套口径
评估计划	按附录 C 开展真实用户任务测试与可用时段数据验证，补充漏检率与效率对比证据

7 参考文献

[1] 张海藩, 牟永敏. 软件工程导论[M]. 6版. 北京: 清华大学出版社, 2013.

[2] 萨师煊, 王珊. 数据库系统概论[M]. 5版. 北京: 高等教育出版社, 2014.

[3] SOMMERVILLE I. 软件工程[M]. 10版. 彭鑫, 赵文耘, 译. 北京: 机械工业出版社, 2018.

[4] Pallets Projects. Flask Documentation[EB/OL]. (2024)[2026-10-16]. https://flask.palletsprojects.com/.

[5] SQLite. SQLite Documentation[EB/OL]. [2026-10-16]. https://www.sqlite.org/docs.html.

[6] openpyxl. openpyxl — A Python library to read/write Excel 2010 xlsx/xlsm files[EB/OL]. [2026-10-16]. https://openpyxl.readthedocs.io/.

[7] Google. Material Design 3[EB/OL]. [2026-10-16]. https://m3.material.io/.

[8] pytest. pytest Documentation[EB/OL]. [2026-10-16]. https://docs.pytest.org/.

[9] Feather. Feather Icons (MIT License)[EB/OL]. [2026-10-16]. https://feathericons.com/.

[10] 全国人民代表大会常务委员会. 中华人民共和国劳动法（2018年修正）[Z]. 1994年发布, 2018年修正.（每周工时上限默认值的参考口径之一）

附录 A　数据库表结构（DDL）

```
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS employees (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    name            TEXT    NOT NULL,
    position        TEXT    NOT NULL DEFAULT '',
    employment_type TEXT    NOT NULL DEFAULT 'fulltime',   -- fulltime / parttime
    phone           TEXT    NOT NULL DEFAULT '',
    available_days  TEXT    NOT NULL DEFAULT '[1,2,3,4,5]', -- JSON：ISO 星期 1–7
    available_from  TEXT    NOT NULL DEFAULT '',            -- HH:MM，空为不限
    available_to    TEXT    NOT NULL DEFAULT '',
    status          TEXT    NOT NULL DEFAULT 'active',      -- active / inactive
    note            TEXT    NOT NULL DEFAULT '',
    created_at      TEXT    NOT NULL DEFAULT (datetime('now','localtime'))
);

CREATE TABLE IF NOT EXISTS shift_templates (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    name          TEXT    NOT NULL,
    start_time    TEXT    NOT NULL,
    end_time      TEXT    NOT NULL,        -- ≤ start 表示次日结束（跨日）
    break_minutes INTEGER NOT NULL DEFAULT 0,
    position      TEXT    NOT NULL DEFAULT '',
    color         TEXT    NOT NULL DEFAULT 'primary',
    note          TEXT    NOT NULL DEFAULT ''
);

CREATE TABLE IF NOT EXISTS schedules (
    id            INTEGER PRIMARY KEY AUTOINCREMENT,
    employee_id   INTEGER NOT NULL REFERENCES employees(id),
    work_date     TEXT    NOT NULL,        -- YYYY-MM-DD（班次开始日期）
    start_time    TEXT    NOT NULL,
    end_time      TEXT    NOT NULL,
    break_minutes INTEGER NOT NULL DEFAULT 0,
    position      TEXT    NOT NULL DEFAULT '',
    template_id   INTEGER REFERENCES shift_templates(id) ON DELETE SET NULL,
    note          TEXT    NOT NULL DEFAULT '',
    created_at    TEXT    NOT NULL DEFAULT (datetime('now','localtime'))
);
CREATE INDEX IF NOT EXISTS idx_schedules_emp_date ON schedules(employee_id, work_date);
CREATE INDEX IF NOT EXISTS idx_schedules_date     ON schedules(work_date);

CREATE TABLE IF NOT EXISTS settings (
    key   TEXT PRIMARY KEY,
    value TEXT NOT NULL
);
```

工时数据不单独建表：由 schedules 按统计周期实时推导（设计决策见 3.3）。规则默认值：最小休息间隔 600 分钟；全职日/周上限 600/2640 分钟；兼职日/周上限 480/1440 分钟——均为演示口径，须按当地法规与机构制度调整。

附录 B　自动化测试用例清单（58 项，2026-10-21 执行全部通过）

B.1 时间计算与冲突检测（test_scheduling.py，26 项）

同日班次跨度；跨日班次跨度；起止相同拒绝；休息扣除；跨日+休息扣除；负休息拒绝；非法时间格式（非两位小时/数值越界/缺冒号）；周区间；月区间（含闰年与12月）；跨日绝对区间展开；无冲突场景；完全重叠；部分重叠含分钟数；首尾相接（休息0+日上限）；跨日与次日重叠；跨日间隔不足；间隔充足无冲突；编辑排除自身；不可排班日；可用时段之外；时段内正常；每日上限（全职）；每日上限（兼职独立口径）；每周上限；周日夜班归属开始周；错误级/提示级区分。

B.2 工时统计（test_worktime.py，6 项）

单班次汇总；跨日班次归属开始日期；周期外排除；周日跨夜班次的周界归属；多人分组与排序；分钟转小时。

B.3 Excel 导出（test_exporter.py，3 项）

排班表标题/表头/跨日时段/行数；工时表逐日列与合计行；空数据健壮性。

B.4 认证与账号（test_auth.py，6 项）

未登录页面重定向与 API 401；登录页渲染与错误密码提示；登录成功跳转 next 与登出；修改密码后旧密码失效；改密三类校验（当前密码/长度/一致性）；登录失败限速（5 次后锁定且正确密码不放行）。

B.5 端到端功能（test_app.py，17 项）

六页面渲染；员工增删与可用时段校验；模板创建（跨日）与起止相同拒绝；冲突 409→强制保存→删除闭环；非法时间 400；演示周冲突检出（含重叠与可用时间）；工时页合计；排班页冲突面板；排班表导出内容与 MIME；工时表逐日列与合计；设置回写与非法值拒绝；停用员工不进入排班网格；备份下载；系统标题设置往返与留空回落；品牌头像上传/伪造文件拒绝/移除恢复默认。（Workers 版另有 13 项 Node 测试：10 项端到端含登录保护/登出/改密，与 3 项 Python↔JS 对齐测试。）

附录 C　需求验证计划（尚未开展，无虚构数据）

受时间与资源限制，以下验证工作在原型阶段未开展。为避免以未执行的调研冒充已完成成果，此处仅列出计划，供后续按此执行：

编号	验证项	方法与对象	记录指标
V1	排班效率对照	邀请 2～3 名有排班经验者，分别用 Excel 与本系统编制同一份“7 天 × 8 人、含 2 处约束冲突”的周表	完成耗时、修改次数、冲突残留数
V2	冲突提示可读性	请使用者对 11 条真实提示逐条评分（能否不看文档理解原因）	5 分制可读性均分、误读条数
V3	工时口径认可度	请使用者核对跨日归属与休息扣除口径是否符合其门店习惯	认可/需调整意见清单
V4	真实数据漏检率	取合作门店 4 周历史班表导入，与门店实际发生的调班/拒班记录比对	系统提示数 vs 实际问题数（查全/查准）
V5	规则默认值调研	收集目标地区同类门店的休息间隔与工时上限惯例，校准默认值	建议值区间及依据

伦理说明：涉及门店数据的验证须在取得经营方书面许可、隐去员工个人信息后进行；问卷与访谈仅使用经参与者允许的内容。

附录 D　演示视频脚本要点（约 3 分 30 秒）

时间	画面	解说要点
0:00–0:25	纸面/Excel 排班照片（示意）	点出四个痛点：冲突靠人眼、约束靠记忆、口径不统一、汇总靠加班
0:25–0:50	概览页（图 4-1）	一周概况一屏可见：班次数、工时、待处理冲突
0:50–1:30	员工页、模板页（图 4-5）	维护可用时间与兼职类型——这是后续检测的数据来源；常用班次沉淀为模板
1:30–2:20	排班页 + 新增对话框（图 4-2、4-3）	点格排班、模板套用；演示为周六休班员工排班 → 冲突对话框（图 4-4），展示“返回修改 / 仍要保存”
2:20–2:50	冲突面板特写	逐条读一条错误与一条提示，强调“给出原因与数字”
2:50–3:20	工时统计 + 导出（图 4-6）	日/周/月切换；打开导出的 xlsx，展示逐日列、合计行与冲突备注
3:20–3:30	设置页（图 4-7）	规则可配置，提醒按当地法规确认设置；结尾重申边界：原型阶段未含员工端

录制要求：全程使用真实运行系统演示（本报告截图即取自该系统），不做静态页面冒充；视频文件按主办方要求的格式与时长提交。